

AI-Powered Student Workload Balancer

Project Proposal for UCS503P
Submitted to: Dr. Nisha Thakur
Thapar Institute of Engineering and Technology

Khushi, CSED* Umang Gautam, CSED[†] Vriti Mahajan, CSED[‡]
Shaurya Garg, CSED[§]

August 18, 2026

1 Higher-order goal

... secondary framing

This project aims to improve student time-management outcomes by reducing the cognitive overhead of planning academic workload across subjects, topics, and deadlines. While the broader goal is student well-being and academic performance, the immediate engineering objective is to translate *adaptive workload planning* into a measurable, deployable software solution.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by prototyping the core planning workflow (input subjects/hours/deadlines → generated plan) and validating it with users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

3 Problem Statement

Students juggling multiple subjects, assignments, and deadlines struggle to allocate limited study time effectively. Common pain points include:

- **Static planning:** manually made schedules quickly become outdated once a student falls behind or a deadline shifts.
- **No feedback loop:** most planning tools do not adapt based on actual progress or performance, leaving students to manually re-plan.

*Roll No: 1024030809, Email: kkhushi1.be24thapar.edu

[†]Roll No: 1024030805, Email: ugautam.be24thapar.edu

[‡]Roll No: 1024030801, Email: vmahajan1.be24thapar.edu

[§]Roll No: 1024030800, Email: sgarg6.be24thapar.edu

- **Uneven workload distribution:** without a systematic priority scheme, students over-invest in familiar subjects and under-invest in weak or high-stakes ones.
- **Opaque reasoning:** even when a tool suggests a plan, students rarely understand *why* that plan was chosen, reducing trust and adoption.

Why this matters now (contextual relevance): With increasing academic load and limited study hours, even small improvements in workload distribution and re-planning speed can noticeably reduce student stress and missed deadlines.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based** “AI-Powered Student Workload Balancer” with the following components:

- **Student input portal:** students enter subjects, topics, available study hours, deadlines, and assignments.
- **Deterministic workload engine:** a rules-based backend service computes a personalized, prioritized study plan from the inputs and performance history.
- **Adaptive feedback loop:** the plan is continuously re-balanced based on actual progress and performance reported by the student.
- **Reasoning agent:** a LangGraph-based agent explains *why* the plan looks the way it does, without altering the underlying business logic.
- **Progress dashboard:** students track plan adherence and upcoming priorities.

4.2 Core workflow

1. Student registers subjects, topics, available hours, and deadlines.
2. The deterministic workload engine computes a prioritized study plan using a scoring algorithm (deadline proximity, topic weight, past performance).
3. Student follows the plan and logs actual progress (completed/missed sessions, performance on topics).
4. The engine re-balances the remaining plan based on this feedback; the agent explains the change in plain language on request.

4.3 Operational constraints for an educational setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Keep all scheduling logic deterministic and auditable; the AI agent may only reason over and explain outputs, never bypass business rules.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on a layered, maintainable system architecture rather than novel algorithm research.

5.1 Deterministic engine + reasoning agent

... *non-research*

Prioritization will be heuristic-based, not black-box:

- Compute a priority score per topic from deadline proximity, estimated effort, and past performance.
- Allocate available study hours across topics in priority order using a pure-function, testable scoring service.
- The LangGraph agent calls this service via tools rather than accessing the database directly, so it is structurally unable to bypass the deterministic business rules – it can only explain the engine’s output, not override it.

5.2 Web architecture

... *web-based solution*

A layered, one-directional web architecture:

- **Frontend (React):** responsive UI for subject/deadline entry, plan viewing, and progress logging.
- **Backend API (FastAPI):** routes → services → repositories → database, a strict one-directional layering enforced structurally rather than by convention.
- **Database (PostgreSQL, via SQLAlchemy ORM):** stores students, subjects, topics, plans, progress logs, and performance history.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

... *measurable and attributable*

We define success using measurable metrics tied to the core adaptive planning workflow.

6.1 Primary evaluation metric

Plan adherence: the percentage of planned study hours/sessions actually completed by the student, as logged in the system.

- Target: median plan adherence $\geq 70\%$ during pilot window.
- Attribution: measured from database timestamps comparing planned sessions vs. logged completions.

6.2 Secondary evaluation metrics

- **Rebalancing responsiveness:** how accurately and quickly the engine redistributes remaining workload after a missed or completed session, measured against remaining time-to-deadline.
- **Deadline miss rate:** percentage of tracked deadlines missed compared to a baseline (self-reported prior habits).
- **User trust/clarity:** student-reported clarity of plan reasoning using a simple 1–5 feedback question after viewing an agent explanation.
- **Reliability:** availability of staging/production for login and plan generation during typical study hours.

6.3 Pilot validation plan

... high-level

- Recruit 10–20 pilot student users across varying course loads.
- Compare plan adherence and deadline-miss rate before and after using the system.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of students, subjects, and concurrent plan re-computation by:
 - decoupling components (frontend/backend/agent),
 - enabling pagination and indexing for common queries,
 - using stateless API design for the deterministic engine.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed PostgreSQL database,
 - containerized or platform-hosted deployment,
 - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs (FastAPI) and reactive frontend (React).
- Managed relational database (PostgreSQL) with a mature ORM (SQLAlchemy).
- Agent orchestration framework (LangGraph) for the explanation layer.
- Authentication approach (token-based).
- Test framework for unit/integration tests (pytest).
- CI runner and staging deployment target.

We will use these mature components to focus on workload-engine correctness, layered architecture, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Subject/topic/deadline input form + student profile
- Deterministic priority-scoring workload engine (pure functions)
- Generated study plan view
- Basic progress logging and timestamps for adherence measurement
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- Continuous feedback loop: re-balancing plan from logged progress
- LangGraph agent for plan explanation via tool calls
- Performance-history-aware prioritization
- Progress dashboard with adherence and rebalancing metrics
- Course workload/difficulty data collected from student feedback to support performance-history-aware prioritization.

10 Risks and mitigations

- **Data privacy / consent:** minimize collected personal data; store only what is needed for planning and evaluation.
- **Student adoption:** keep input workflow minimal; provide sensible defaults so a first plan can be generated quickly.
- **Evaluation measurement ambiguity:** instrument timestamps and define clear session/completion states before development begins.
- **Agent overreach:** enforce strict tool-based access so the agent cannot bypass the deterministic engine's business rules.

11 Summary

This project proposes a deployable web platform that generates and continuously adapts a personalized student study plan. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially plan adherence). It remains focused on engineering workflow, layered architecture, and structural safety of the AI reasoning layer rather than research-driven novelty.